جامعة أم القرى
UMM AL-QURA UNIVERSITY



Amusement park

DATABASE PROJECT

REQUIREMENT COLLECTION AND ANALYSIS



EMPLOYEES:

- Each employee must have a unique EmployeeID.
- Employees must have a name, phone number, assigned roles, position, and salary.
- Age and gender are optional information.
- An employee can manage multiple games, tickets, and restaurants.
- An employee can take multiple food orders.



GAMES:



- Each game must have a unique GameID.
- Games must have a name, price, duration, age restriction, and scheduled time.
- A game is assigned to a specific employee.
- Visitors can only play games that are within their age restriction.
- Games may have scheduled times, but visitors can choose to play the game at any time within operating hours.



TICKETS:

- Each ticket must have a unique TicketID.
- Tickets must have a type, price, assigned employee, and valid date range.
- A visitor can only have one ticket at a time.
- Tickets can be used by multiple visitors within their valid date range.





RESTAURANTS:

- Each restaurant must have a unique RestaurantID.
- Restaurants must have a name and cuisine type.
- A restaurant is assigned to a specific employee.
- Visitors can place food orders at restaurants within the park.



VISITORS:

- Each visitor must have a unique VisitorID.
- Visitors must have a name and age.
- Health conditions are optional information.
- A visitor must have a ticket to enter the park.
- A visitor can play multiple games within the park.





VISITORSGAME:

- A visitor can play multiple games, but each visitor-game combination must be unique.
- VisitorsGame must include the (VisitorID, GameID)



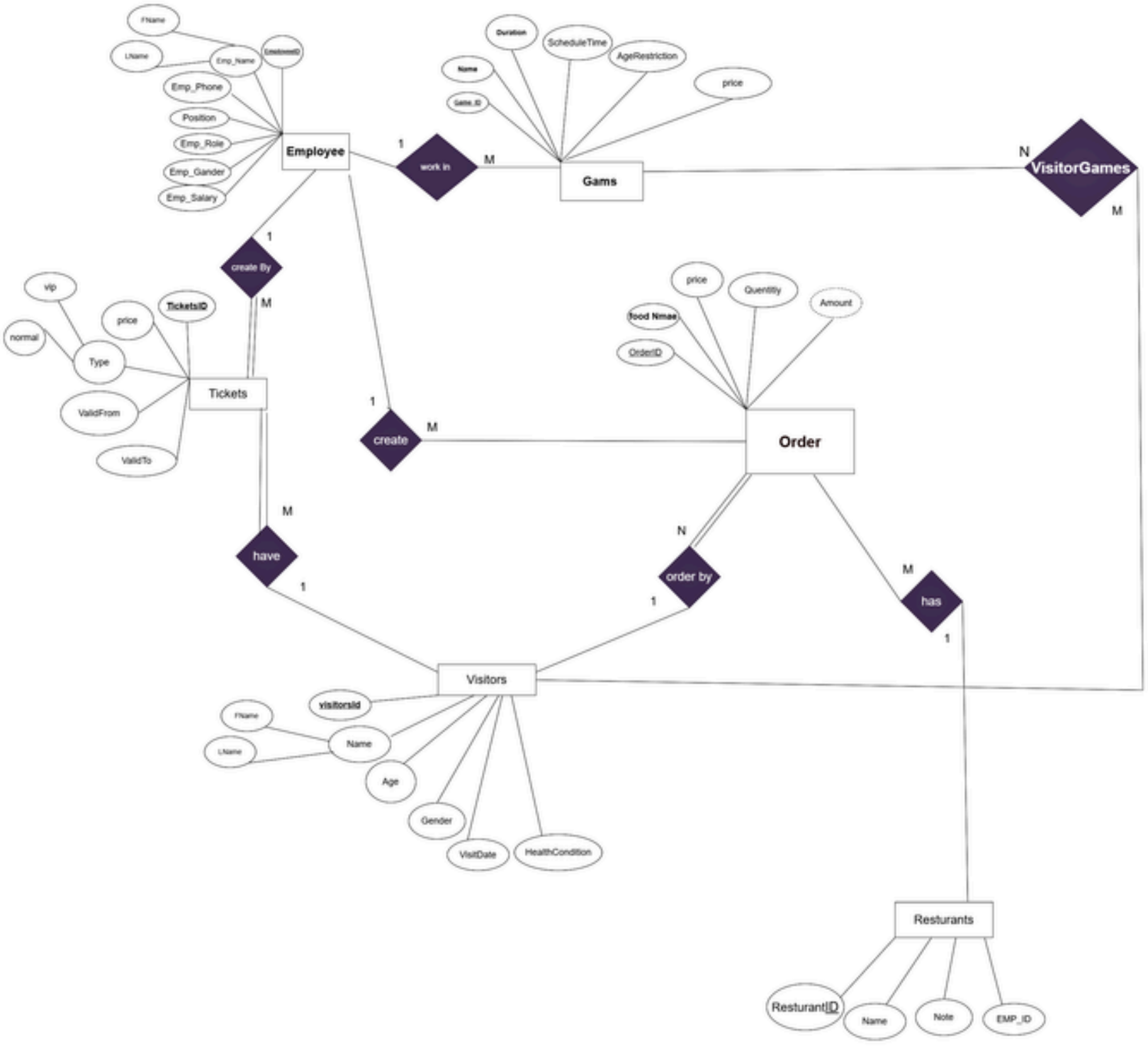
ORDERS:

- Each order must have a unique OrderID.
- Orders must include the (food name, price, quantity), and assigned employee.
- An order is associated with a specific restaurant and a specific visitor.

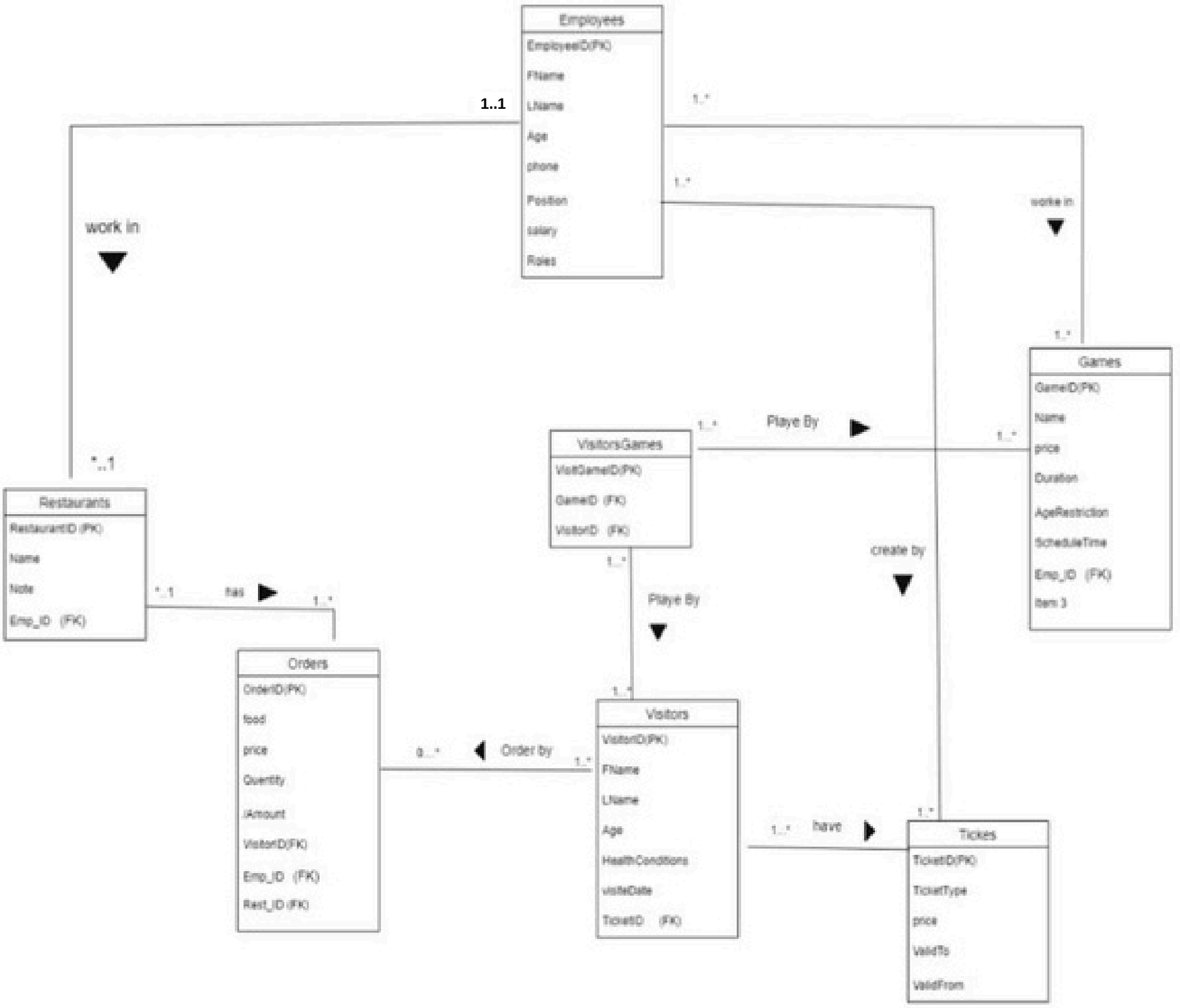




ER MODEL IN TWO NOTATIONS: - CHEN'S NOTATION



UML NOTATION



Mapping Relational Schema

The Er Model is Mapped to the following Relational Schema

Step 1: Mapping Of Regular Entity Type:

1.1 Tickers

TicketsID	TicketType	price	ValidFrom	ValidTo

1.2 Games

GameID	Name	Duration	ScheduleTime	AgeRestriction

1.3 Employees

EmployeeID	Emp_ FName	Emp_Lname	Emp_ Age	Emp_ Salary	Emp_ phone	Emp_ Position	Emp_ Role

1.4 Orders

OrderID	FoodName	price	Quentitiy	Amount	

1.5 Visitors

VisitorID	FName	Lname	Age	Gender	phone	VisitDate	HealthConditions

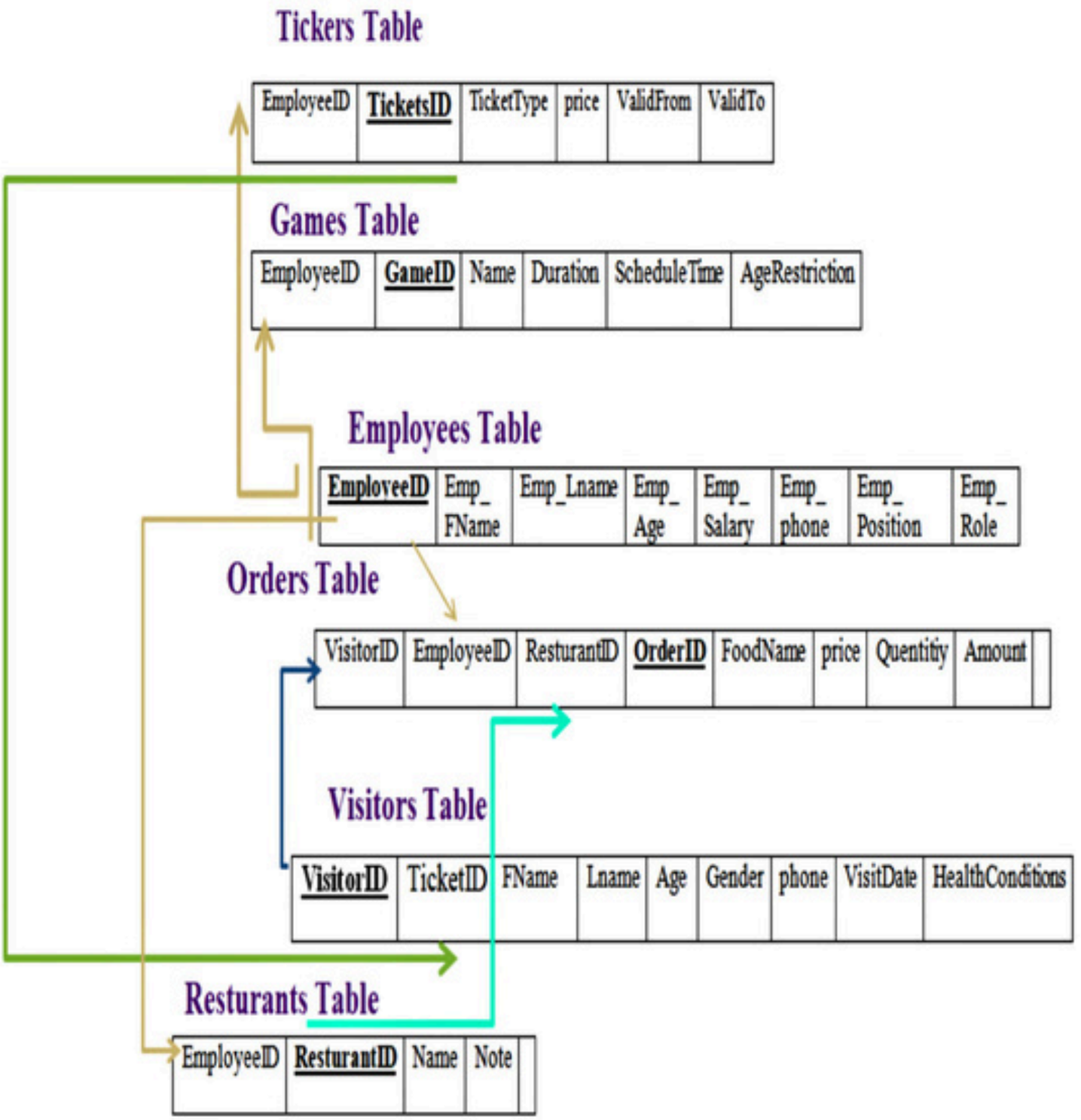
1.6 Resturants

ResturantID	Name	Note	

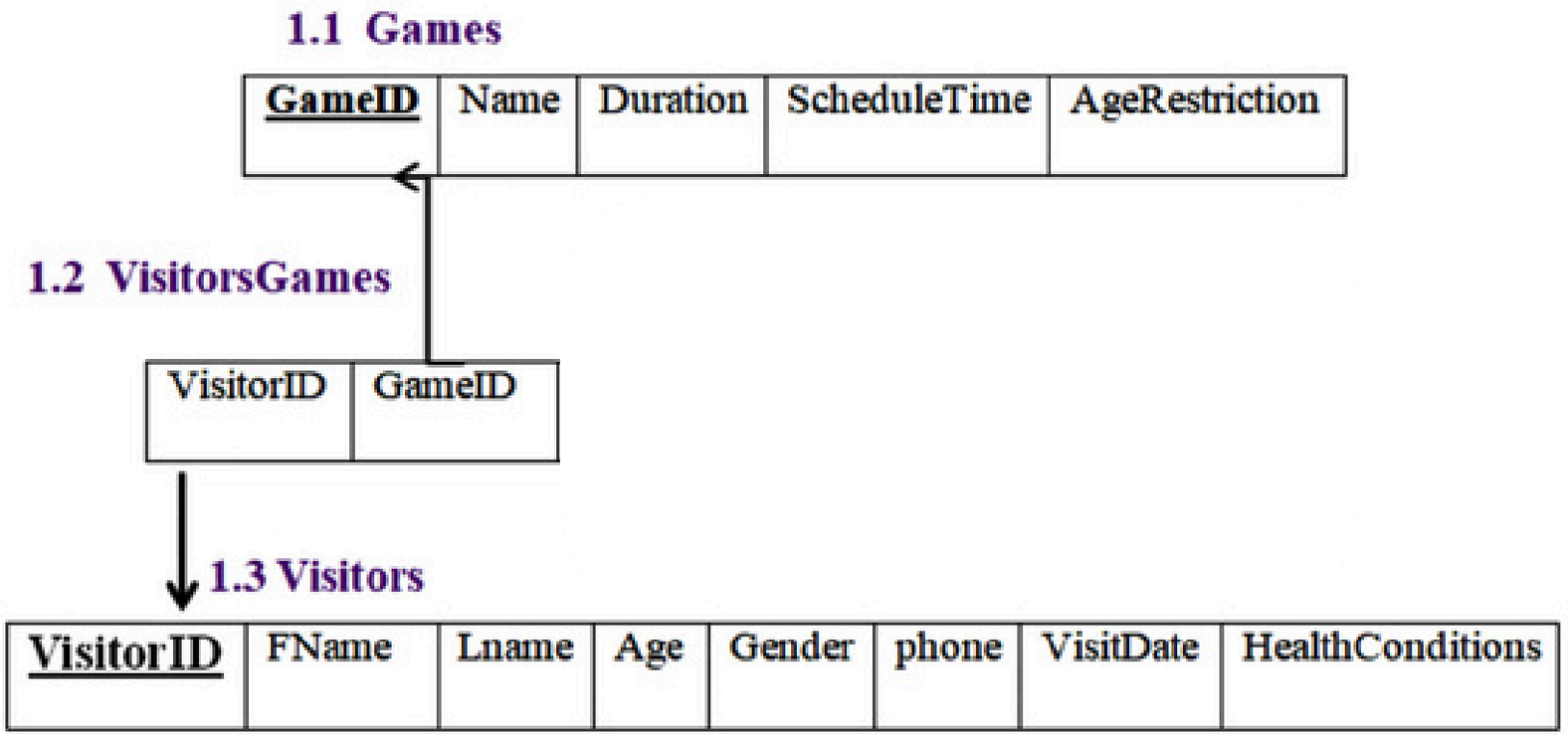
Step 2: There is No Mapping of Weak Entity Types.

Step 3: There is No Mapping of Binary 1:1 Relationship Types.

Step 4: There is Mapping of Binary 1:N Relationship Types.



Step 5: There is Mapping of Binary M:N Relationship Types.

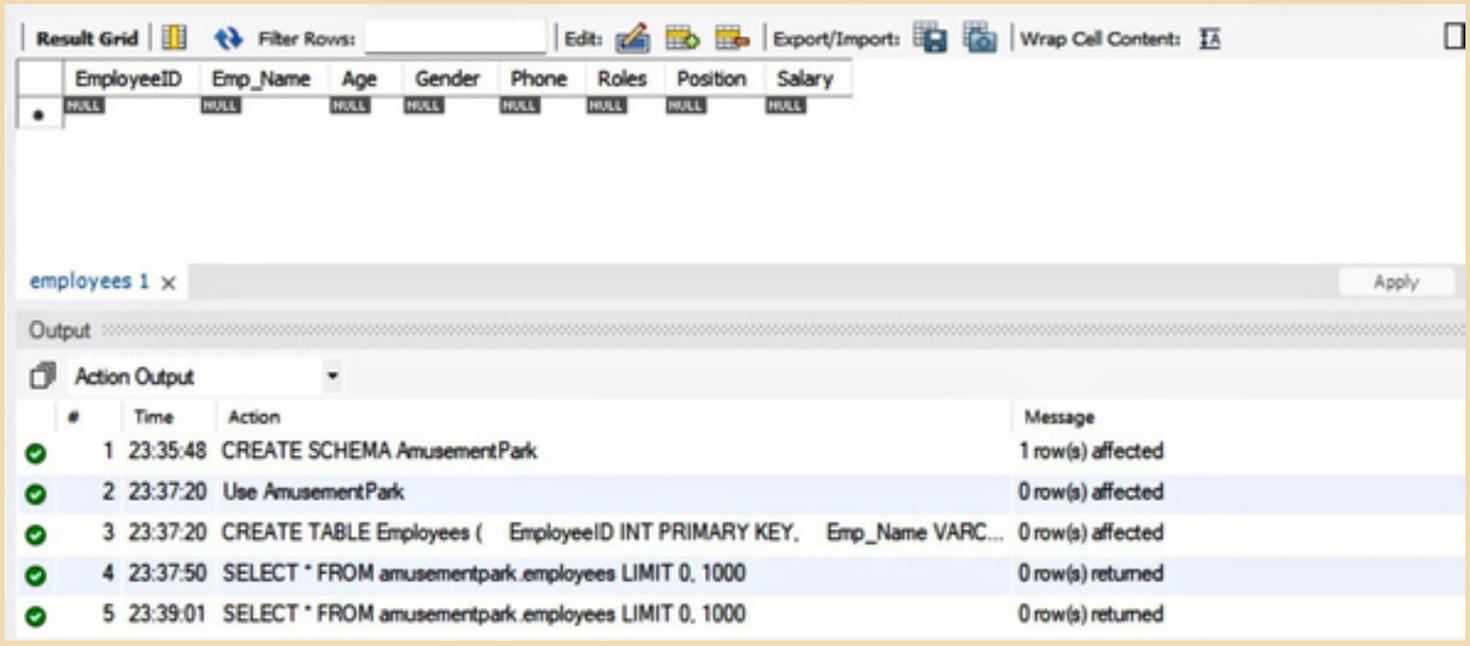


Step 6: There is No Mapping of Multiivalued Attributes.

SQL Commands and Results

1- CREATE TABLE Employees

```
CREATE TABLE Employees (
        EmployeeID INT PRIMARY KEY,
        Emp_Name VARCHAR(100) NOT NULL,
          Age int NULL,
          Gender VARCHAR(10) NULL,
          phone VARCHAR(20) NOT NULL,
          Roles VARCHAR(20) NOT NULL,
        Position VARCHAR(50) NOT NULL,
        salary decimal (10,2) NOT NULL,
);
```



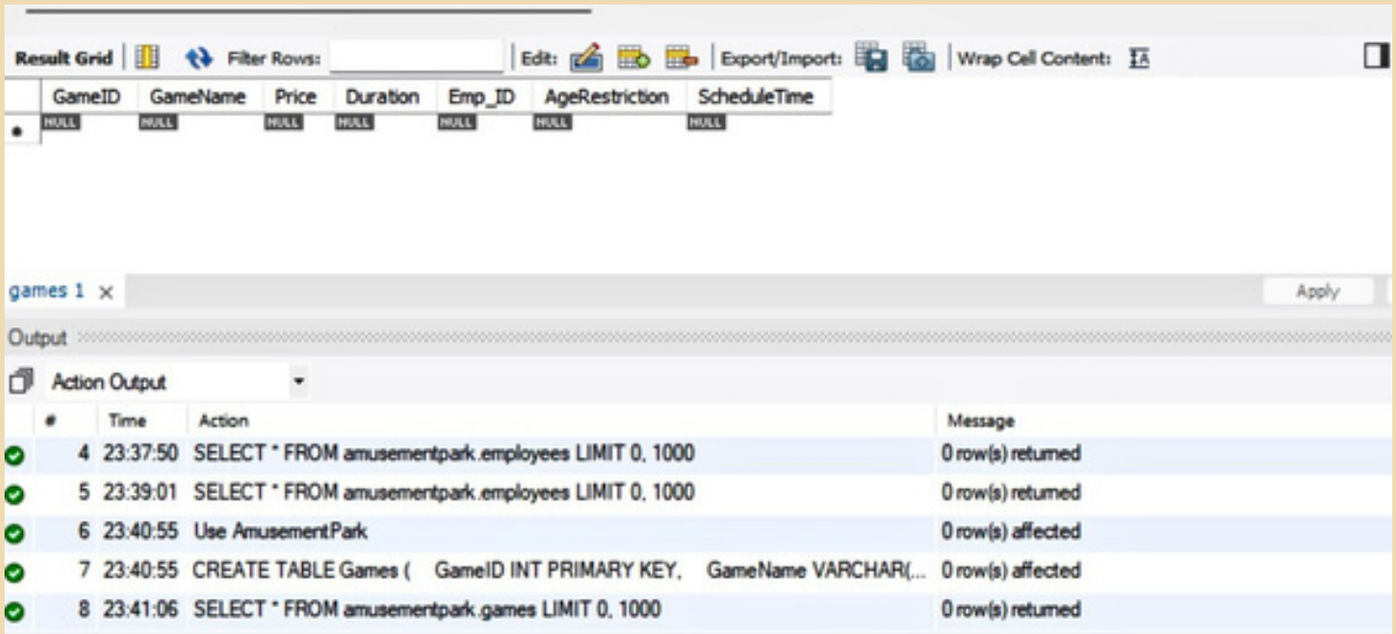






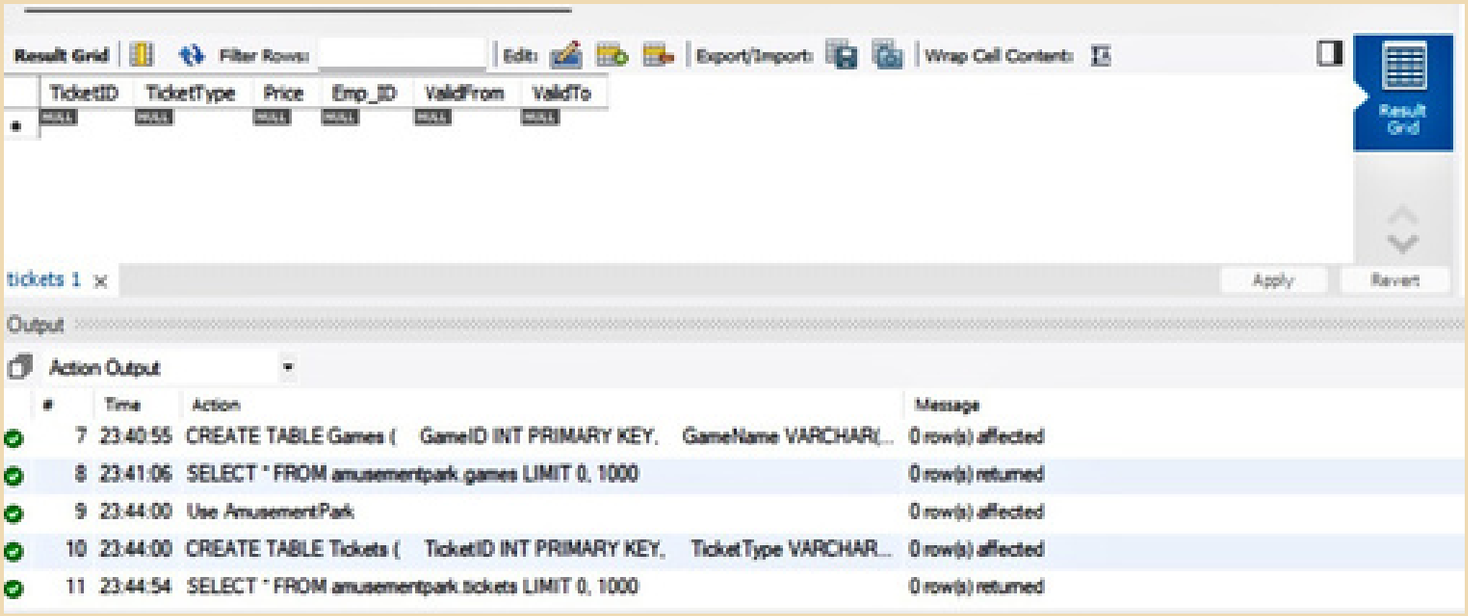

2- CREATE TABLE GAMES

```
CREATE TABLE Games (
        GameID INT PRIMARY KEY,
        GameName VARCHAR(100) NOT NULL,
          price Disimal (10,2) NOT NULL,
        Duration INT,
          Emp_ID int,
          AgeRestriction INT,
          ScheduleTime TIME,
          FOREIGN KEY (Emp_ID) REFERENCES Employees(EmployeeID)
);
```



3-CREATE TABLE TICKETS

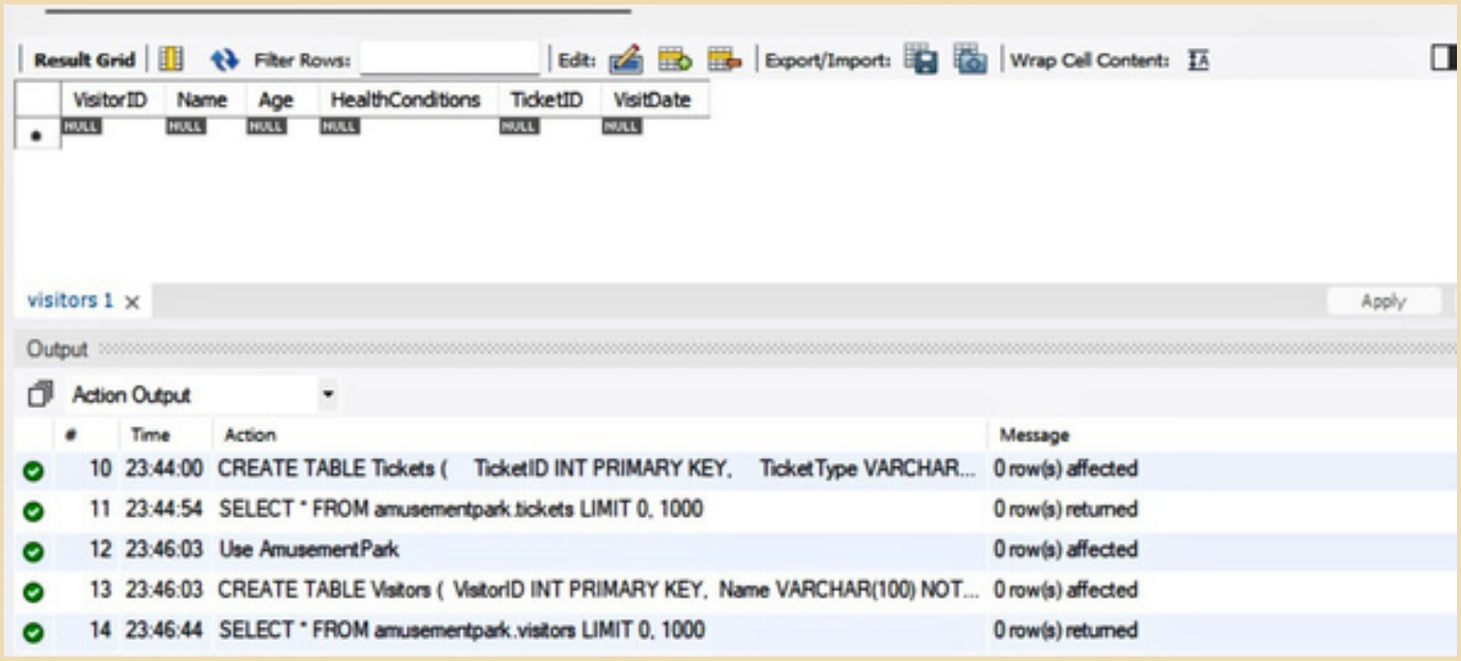
```
CREATE TABLE Tickets (
        TicketID INT PRIMARY KEY,
        TicketType VARCHAR(50) NOT NULL,
        Price DECIMAL(10,2) NOT NULL,
           Emp_ID INT,
        ValidFrom DATE,
        ValidTo DATE,
       FOREIGN KEY ( Emp_ID) REFERENCES Employees (EmployeeID)
);
```



4- CREATE TABLE VISITORS

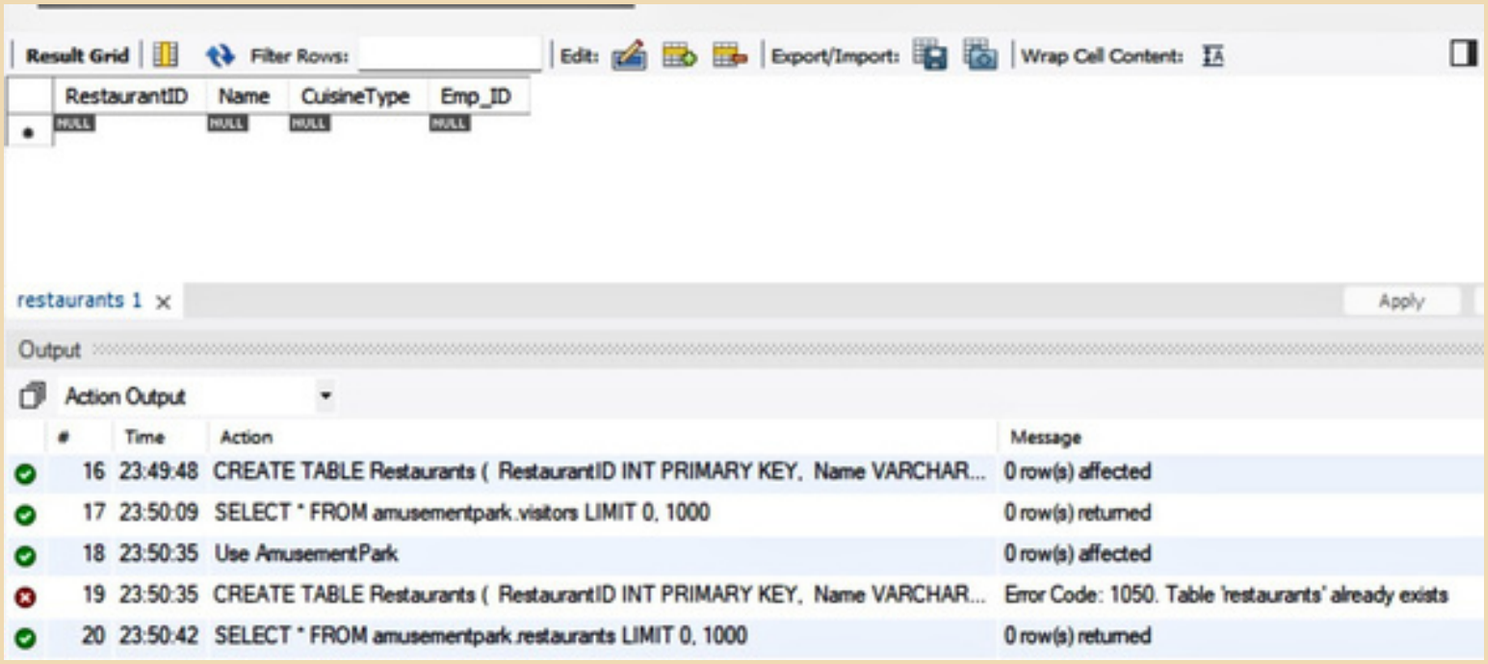
```
CREATE TABLE Visitors (
        VisitorID INT PRIMARY KEY,
        Name VARCHAR (100) NOT NULL,
       Age INT NULL,
          HealthConditions VARCHAR (50) NULL,
       TicketID INT,
       VisitDate DATE,

    FOREIGN KEY (TicketID) REFERENCES Tickets (TicketID)
);
```



5- CREATE TABLE RESTAURANTS

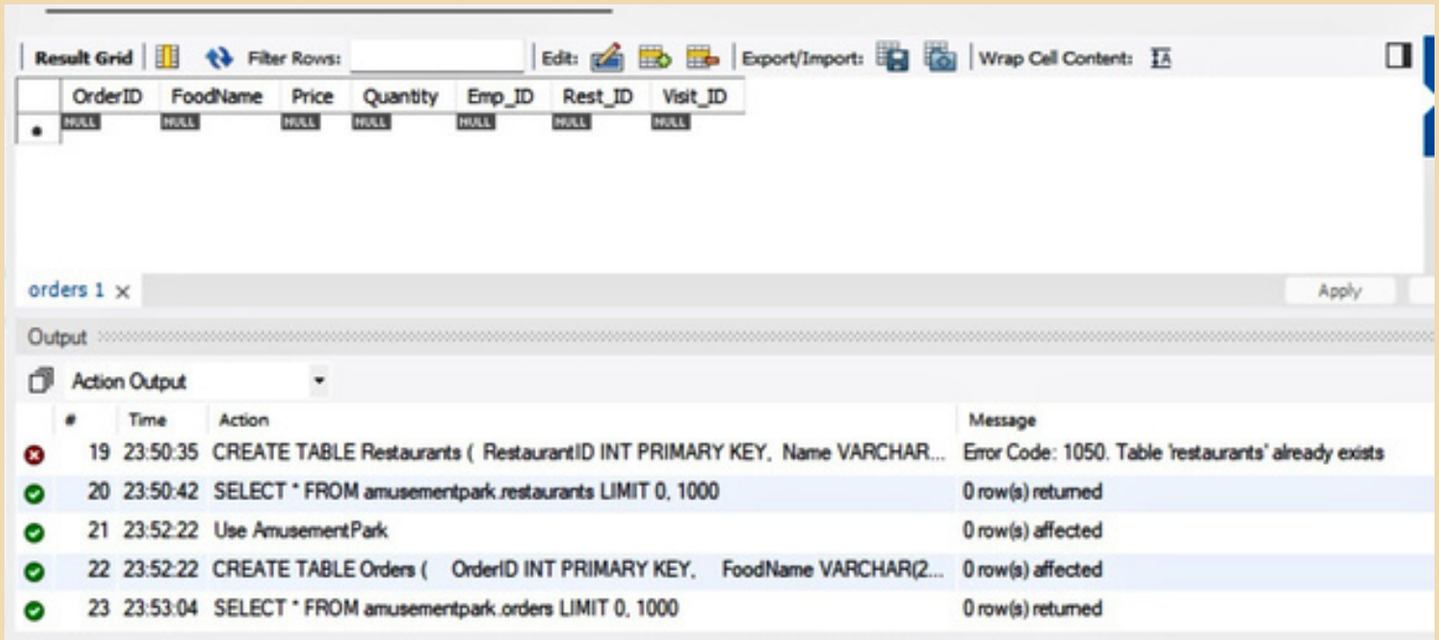
```
CREATE TABLE Restaurants (
  RestaurantID INT PRIMARY KEY,
  Name VARCHAR (100) NOT NULL,
  CuisineType VARCHAR (50),
  Emp_ID int,
  FOREIGN KEY (Emp_ID) REFERENCES Employees (EmployeeID)
);
```



6- CREATE TABLE ORDERS

```
CREATE TABLE Orders (

        OrderID INT PRIMARY KEY,
        FoodName VARCHAR (20),
            Price DECIMAL (10,2),
        Quantity int ,
        Emp_ID int,
        Rest_ID int,
            Visit_ID INT ,
  FOREIGN KEY (Rest_ID) REFERENCES Restaurants (RestaurantID),
  FOREIGN KEY (Emp_ID ) REFERENCES Employees (EmployeeID),
  FOREIGN KEY (Visit_ID ) REFERENCES Visitors (VisitorID),
);
```

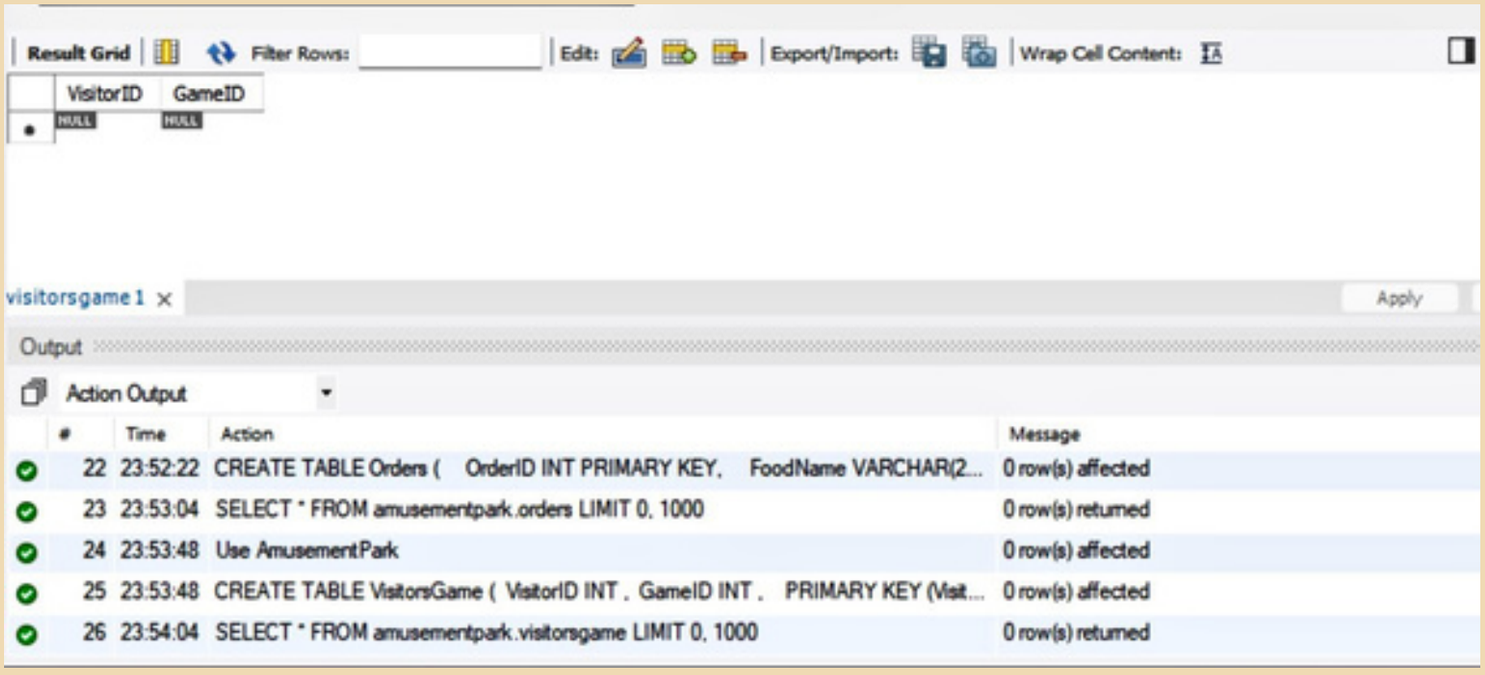


7- CREATE TABLE VisitorsGames

```
CREATE TABLE VisitorsGame (

        VisitorID INT,
        GameID INT,

        PRIMARY KEY (VisitorID, GameID),
        FOREIGN KEY (VisitorID) REFERENCES Visitors (VisitorID),
        FOREIGN KEY (GameID) REFERENCES Games (GameID)
);
```



Data Insertion



1-

```sql
INSERT INTO Employees VALUES
(101, 'Ali', 30, 'Male', '0555123456', 'Manager', 'Admin', 5000.00),
(102, 'Nourah', 25, 'Female', '0555123457', 'Cashier', 'Sales', 3000.00),
(103, 'Sara', 35, 'Female', '0555123458', 'Supervisor', 'Operations', 4000.00),
(104, 'Ahmed', 28, 'Male', '0555123459', 'Technician', 'Maintenance', 3500.00);
```



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

EmployeeID	Emp_Name	Age	Gender	phone	Roles	Position	salary
101	Ali	30	Male	0555123456	Manager	Admin	5000.00
102	Nourah	25	Female	0555123457	Cashier	Sales	3000.00
103	Sara	35	Female	0555123458	Supervisor	Operations	4000.00
104	Ahmed	28	Male	0555123459	Technician	Maintenance	3500.00
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

employees 1 ×

Output



2-

```sql
INSERT INTO `games` VALUES
 (1,'Laser Tag',20.00,60,101,10,'14:00:00'),
(2,'Bowling',15.00,90,101,5,'16:00:00'),
(3,'Escape Room',25.00,120,102,12,'18:00:00');
```



Result Grid | Filter Rows: | Edit: | Export/Import:

GameID	GameName	price	Duration	Emp_ID	AgeRestriction	ScheduleTime
1	Laser Tag	20.00	60	101	10	14:00:00
2	Bowling	15.00	90	101	5	16:00:00
3	Escape Room	25.00	120	102	12	18:00:00
NULL	NULL	NULL	NULL	NULL	NULL	NULL





3-

```
INSERT INTO `tickets` VALUES
(1,'Adult',50.00,101,'2024-01-01','2024-12-31'),
(2,'Child',25.00,102,'2024-01-01','2024-12-31'),
(3,'FamiluPackage',250.00,103,'2024-01-01','2024-12-31'),
(4,'Student',35.00,104,'2024-01-01','2024-12-31');
```

Result Grid | Filter Rows: | Edit: | Expo

TicketID	TicketType	Price	Emp_ID	ValidFrom	ValidTo
1	Adult	50.00	101	2024-01-01	2024-12-31
2	Child	25.00	102	2024-01-01	2024-12-31
3	FamiluPackage	250.00	103	2024-01-01	2024-12-31
4	Student	35.00	104	2024-01-01	2024-12-31
NULL	NULL	NULL	NULL	NULL	NULL

4-

```
INSERT INTO `restaurants` VALUES
(1,'Pizza Palace','Italian',101),
(2,'Sushi World','Japanese',104),
(3,'Grill Master','American',102),
(4,'Taco Fiesta','Mexican',102);
```

RestaurantID	Name	CuisineType	Emp_ID
1	Pizza Palace	Italian	101
2	Sushi World	Japanese	104
3	Grill Master	American	102
4	Taco Fiesta	Mexican	102
NULL	NULL	NULL	NULL








5-

```
INSERT INTO `orders` VALUES
(1,'Burger',15.00,3,102,1,1),
(2,'Pizza',25.00,1,103,2,2),
(3,'Sushi',30.00,2,102,3,3),
(4,'Tacos',10.00,4,103,4,4);
```

OrderID	FoodName	Price	Quantity	Emp_ID	Rest_ID	Visit_ID
1	Burger	15.00	3	102	1	1
2	Pizza	25.00	1	103	2	2
3	Sushi	30.00	2	102	3	3
4	Tacos	10.00	4	103	4	4
NULL	NULL	NULL	NULL	NULL	NULL	NULL

6-

```
INSERT INTO `visitors` VALUES
(1,'Ali',22,'None',1,'2024-10-01'),
(2,'Mona',30,'Asthma',2,'2024-10-02'),
(3,'Yousef',18,'Diabetes',3,'2024-10-03'),
(4,'Laila',25,'None',4,'2024-10-04');
```

Result Grid | Filter Rows: | Edit:

VisitorID	Name	Age	HealthConditions	TicketID	VisitDate
1	Ali	22	None	1	2024-10-01
2	Mona	30	Asthma	2	2024-10-02
3	Yousef	18	Diabetes	3	2024-10-03
4	Laila	25	None	4	2024-10-04
NULL	NULL	NULL	NULL	NULL	NULL

7-

```
INSERT INTO `visitorsgame` VALUES
(2,1),(4,1),(3,2),(1,3);
```

	VisitorID	GameID
▶	2	1
	4	1
	3	2
	1	3
*	NULL	NULL








The Commands

- List of all salaries higher than 3500

```
SELECT * FROM Employees WHERE salary > 3500;
```

EmployeeID	Emp_Name	Age	Gender	phone	Roles	Position	salary
101	Ali	30	Male	0555123456	Manager	Admin	5000.00
103	Sara	35	Female	0555123458	Supervisor	Operations	4000.00

- Select Data “Game”

```
SELECT * FROM Games;
```

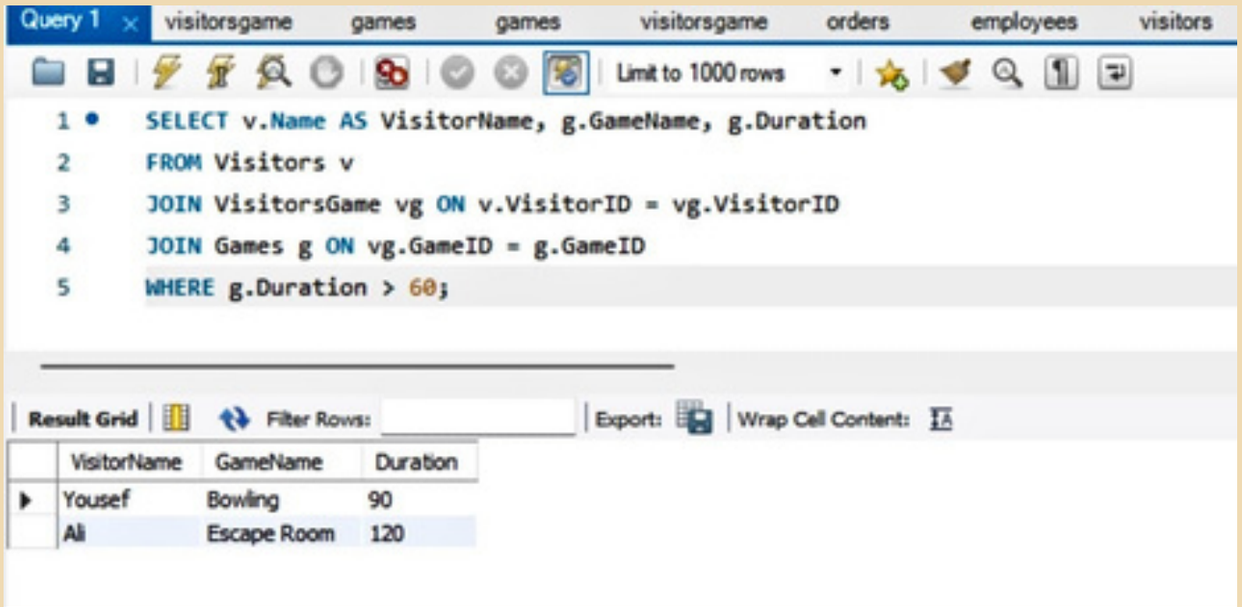
GameID	GameName	price	Duration	Emp_ID	AgeRestriction	ScheduleTime
1	Laser Tag	20.00	60	101	10	14:00:00
2	Bowling	15.00	90	101	5	16:00:00
3	Escape Room	25.00	120	102	12	18:00:00

- Select from “games” names sorted by price in ascending order

```
SELECT GameName, price
FROM Games
ORDER BY price ASC;
```

GameName	price
Bowling	15.00
Laser Tag	20.00
Escape Room	25.00

- List of people who played for more than 60 minutes



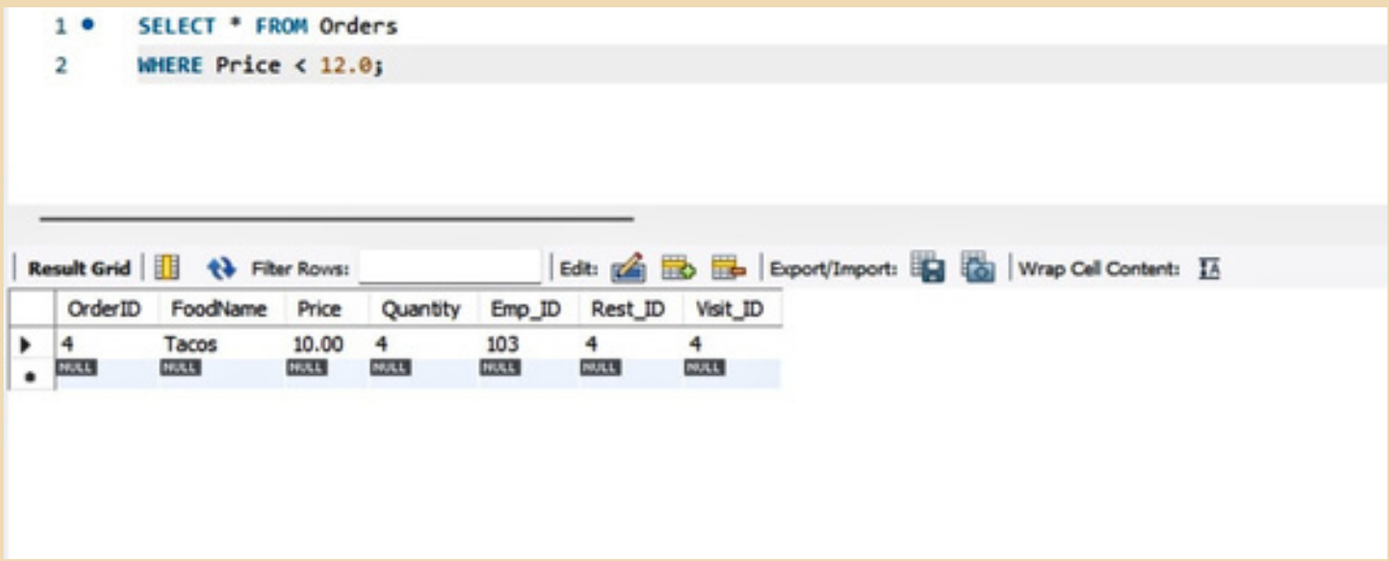
```
Query 1 | visitorsgame | games | games | visitorsgame | orders | employees | visitors
Limit to 1000 rows

1  SELECT v.Name AS VisitorName, g.GameName, g.Duration
2  FROM Visitors v
3  JOIN VisitorsGame vg ON v.VisitorID = vg.VisitorID
4  JOIN Games g ON vg.GameID = g.GameID
5  WHERE g.Duration > 60;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

VisitorName	GameName	Duration
Yousef	Bowling	90
Ali	Escape Room	120

- List of food prices under 12.0



```
1  SELECT * FROM Orders
2  WHERE Price < 12.0;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

OrderID	FoodName	Price	Quantity	Emp_ID	Rest_ID	Visit_ID
4	Tacos	10.00	4	103	4	4
NULL	NULL	NULL	NULL	NULL	NULL	NULL

1– Update Data Employees

```sql
UPDATE Employees
SET Emp_Name = 'Khalid', Age = 36, Gender = 'Male', phone = '0555689876', Roles = 'Supervisor',
Position = 'Operations Manager', salary = 75000.00
WHERE EmployeeID = 103;
```

The Output :

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

EmployeeID	Emp_Name	Age	Gender	phone	Roles	Position	salary
101	Ali	30	Male	0555123456	Manager	Admin	5000.00
102	Nourah	25	Female	0555123457	Cashier	Sales	3000.00
103	Khalid	36	Male	0555689876	Supervisor	Operations Manager	75000.00
104	Ahmed	28	Male	0555123459	Technician	Maintenance	3500.00
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

2- DELETE to remove orders with a price less than 10:

```sql
Use AmusementPark;
DELETE FROM Orders
WHERE price > 10;
```

The Output :

Result Grid | Filter Rows: | Edit: | Ex

OrderID	FoodName	Price	Quantity	Emp_ID	Rest_ID	Visit_ID
4	Tacos	10.00	4	103	4	4
NULL	NULL	NULL	NULL	NULL	NULL	NULL

3- Group By the average salary for each job (role) in the employee table.

```
2 •   SELECT Roles, AVG(salary) AS AverageSalary
3     FROM Employees
4     GROUP BY Roles;
```

Result Grid | Filter Rows: | Export:

Roles	AverageSalary
Manager	5000.000000
Cashier	3000.000000
Supervisor	75000.000000
Technician	3500.000000

4- HAVING Show job titles with average salaries over 4000

```
3     FROM Employees
4     GROUP BY Roles
5     HAVING AVG(salary) > 4000;
```

Result Grid | Filter Rows:

Roles	AverageSalary
Manager	5000.000000
Supervisor	75000.000000

5- Subqueries Display data for employees who earn salaries higher than the average of all salaries in the Employees table

```
2 •   SELECT Emp_Name, salary
3     FROM Employees
4     WHERE salary > (SELECT AVG(salary) FROM Employees);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conten

	Emp_Name	salary
▶	Khalid	75000.00

The end !

Thank you.

